

# FINM3422 - Coding Project Part 2

## Scenario

You're employed as a Graduate Analyst on the Exotic Derivatives Trading Desk at a leading Sydney-based investment bank. Your team structures and prices bespoke Over-the-Counter (OTC) equity options for institutional clients. You and 2-3 other Graduate Analysts have been placed in a team to write new software for the bank to help modernise their operations.

Your team's first assignment for the new software is to aid the bank in valuing and managing the following specific OTC option positions. All valuations are to be made as of the close of market on Friday, 16 May 2025:

- The bank has been asked to **write a European call option on BHP Group Ltd (BHP)**, with the strike price to be set at **98% of BHP's closing price** on 16 May 2025, and an expiry date of **15 Sept 2027**.
- A client wishes for the bank to **purchase an American put option on Commonwealth Bank of Australia (CBA)**, with a fixed strike price of **\$170.00**, expiring on **15 May 2026**.
- The bank has also been asked to **purchase a European up-and-in barrier call option on Wesfarmers Limited (WES)**. This option will have a strike price of **\$80.00**, an in-barrier level of **\$100.00**, and will expire on **15 Sept 2027**.
- Finally, the bank needs to **write a European basket call option** expiring on **17 July 2025**, with a strike price of **\$175.00** on the basket. The underlying basket is composed of four ASX-listed stocks with the following weights: **10% BHP Group Ltd (BHP)**, **35% CSL Limited (CSL)**, **15% Woodside Energy Group Ltd (WDS)**, and **40% Macquarie Group Ltd (MQG)**.

## Task Description

You and your team have to develop a Python notebook (.ipynb) that uses Object Oriented programming to calculate a price that the bank should offer for each of the four trades given in the task scenario, and that also provides hedging information for the resulting portfolio containing all four positions.

This is designed to be an open-ended task. As well as being graded on the quality of your code and the accuracy of your calculations, a component of your mark will be awarded for the critical thought demonstrated in the structure of your solution.

A minimum viable solution will earn a grade of around 70%, and would need to:

- Define classes for each of the types of derivatives in the scenario.
- Define a method for each of those classes that can value an instance of that class for trading by the bank.
- Provide some method for calculating basic hedge information (e.g. output one hedge parameter) combined portfolio of derivatives.
- Instantiate each of the trades in the scenario using sourced data as at 16 May and calculate the trade prices and hedging parameter(s).

A high-scoring solution would need to demonstrate that careful thought has been put into:

- The Object Oriented architecture of your software - you should design an appropriate inheritance and relationship structure between your classes that minimises code duplication and that helpfully abstracts the use of your software from the detailed calculations contained in the classes.
- How discount rates for future cashflows at different maturities are being calculated.
- The use of valuation methods that are most appropriate for each different type of derivative.
- The consistency of your methods - if a method is defined on one of the classes in your software, then you should also define it for any other classes that a user might intuitively expect it to be defined for.
- The breadth and usefulness of the portfolio hedging information produced by your software.

## **Preparation**

Developing a solution for this task will require you to do some outside research and sourcing of data. Some design decisions, such as the valuation methods to use for each class or the best source instruments to calculate your discount factors from, will need to be based on your own research. You are welcome to make use of generative AI in guiding your research process, and I recommend that you do so to make the process much more efficient. Just make sure to apply your own critical thought throughout the research process or you may end up heading down a dead-end path because your AI was missing some key context about the task.

The data you need to source will depend on how you implement your solution. Some useful sources to check out are the yfinance API, the ASX website, and the UQ Bloomberg terminal (39-106).

Your team should appoint 1 code owner ("CO") who will create a new **private** repo.

You might find that your solution is able to re-use some of the code from the [howesrichard](#) GitHub account. You are welcome to copy code from any of these repos into the CO's own GitHub repo, but **do not fork any of the howesrichard repos** to get started, as you won't be able to make your forked repo private.

## Collaboration

Once the CO has created a new private repo, they should then invite the other members of the team as collaborators. I suggest that the CO creates a branch rule in settings (as demonstrated in our seminar) which requires approval from code owners.

Collaborators will each create a branch (I suggest with your name or account name in the branch name) where your work will live. Collaborators will need to ensure that they're keeping their branch up to date with any changes made to the main branch.

Collaborators will make pull requests to the CO to incorporate changes. I suggest doing this a number of times to incorporate progress. For example if your work is a function that returns something to be used in the primary .ipynb or a .py element of the software, then start by creating a simpler function that returns dummy data or content of the form you plan to implement and doing a pull request for this. That way the CO can incorporate your function (with dummy data) into the software. Then you can refine and improve your function through time.

Start by getting together as a group and planning out what you want the functionality to be. Agree on an implementation roadmap that starts with low hanging fruit so that you know you can deliver a MVP (minimum viable product).

Assign roles to collaborators to build specific elements. Give thought to how this will be co-ordinated. It works best when collaborators are not working on the same files (for obvious, version-control reasons). It also works best when you have agreed inputs and outputs for functions, or agreed attributes and methods for objects (Classes). In other words, get the architecture agreed before you start coding in the actual functionality.

You only have two weeks to complete this task, so you'll need to start planning together quickly so that each member of the team can be working on their assigned components at the same time and you can avoid having any of your team members waiting on other parts of the software to be finished before they start, which will waste some of the limited person-hours available to you.

## Repo Structure

You can set your repo up in whatever way you think is most appropriate for the task, but you need to make sure that it includes the following:

- You should have one main program in a Jupyter notebook named *\*trading\_desk\_analysis.ipynb\**. The grader should be able to navigate immediately to this notebook, open it, and understand how to run the code inside to value the derivative trades and produce hedging parameters for the portfolio.
- You should break your main program down into sections of code blocks that handle different distinct components of your solution. These sections should be separated by markdown cells that use sub-headings and descriptions to guide the user through each section. The descriptions should explain how the section works, and should also summarise and justify the design decisions you made in developing that part of the solution.
- Any user-defined functions you call in your main .ipynb program should be written in one or more external .py files and imported into your main program.

## Individual Marks

Individual marks will be modified based on the total quality of each team member's contribution as recorded by their commits to the group's repo.

## Submission Details

The assignment will be due on Thursday, 29th May at 4pm.

To submit:

- Make sure all of your files are in a single folder. This folder should include:
  - Your main notebook file named `trading_desk_analysis.ipynb`.
  - Any .py files you used to store functions for importing to your main program.
  - Any other files that your main program needs to draw from. Whether there are additional files you need to include will depend on the design of your solution.
  - **Your ".git" hidden folder.** If you haven't set hidden folders to be viewable, just make sure you're zipping the same folder you cloned from your GitHub repo, since this will include the ".git" folder that contains the history of your git commits.
- Zip your folder (How to: [Windows](<https://support.microsoft.com/en-au/windows/zip-and-unzip-files-8d28fa72-f2f9-712f-67df-f80cf89fd4e5>) | [Mac](<https://support.apple.com/en-au/guide/mac-help/mchlp2528/mac>))

- **If you are the team leader**, submit your zip file to the [submission link on Blackboard]([https://learn.uq.edu.au/webapps/assignment/uploadAssignment?content\\_id=\\_11261774\\_1&course\\_id=187907\\_1&group\\_id=&mode=cpview](https://learn.uq.edu.au/webapps/assignment/uploadAssignment?content_id=_11261774_1&course_id=187907_1&group_id=&mode=cpview)). Only one person per group needs to submit, so if you aren't the team leader, you don't submit anything.
- When submitting, **include the URL to your GitHub repo in your submission text**. Invite all three of the course tutors and the course coordinator as collaborators to your GitHub repo: ben-forbes-1, DDB0404, shaunandersen, and JonJAster.